PHASE 2 · SECTION 04

Implementation Details

UniMentor / Software Engineering · Project structure, modularity, dependency management with npm, core integration, quality assurance and deployment.

4.1 Project Structure

This section explains how the code base is organised, why each piece of code lives where it does, how dependencies are managed with npm (the Node.js equivalent of Maven or Gradle), and how frontend, backend, database, quality checks and deployment are wired together. It implements the Layered Architecture of Section 3.1 and ADRs 1, 2, 3, 4 and 7.

4.1.1 Repository Layout

UniMentor is a single Git repository (*monorepo*) with two independently deployable applications, plus documentation and infrastructure files:

```
UniMentor/
├── .github/workflows/ci.yml   CI: lint + tests + audit on every push/PR to main
├── render.yaml                Render Blueprint: API + PostgreSQL (ADR 7)
├── docs/                      Documentation by phase (+ build/ for the PDFs)
├── backend/                   REST API, Node.js + Express (ADR 1)
│   ├── package.json           Manifest: metadata, scripts, dependencies
│   ├── package-lock.json      Exact dependency tree (generated by npm)
│   ├── eslint.config.js       Static analysis rules
│   ├── .env.example           Template of the environment variables
│   ├── src/
│   │   ├── server.js          Entry point: listen() and graceful shutdown
│   │   ├── app.js             Express app: middleware chain and routers
│   │   ├── config/            env.js (validated settings), db.js (PostgreSQL pool)
│   │   ├── routes/            URL -> controller mapping (index.js mounts every router)
│   │   ├── controllers/       HTTP layer (req/res)
│   │   ├── services/          Business rules
│   │   ├── repositories/      SQL and data access
│   │   ├── middlewares/       Cross-cutting: notFound.js, errorHandler.js
│   │   └── utils/             AppError.js (error carrying an HTTP status)
│   └── tests/                 Jest + Supertest suites
└── frontend/                  Static site, vanilla HTML/CSS/JS (ADR 3)
    ├── index.html
    ├── css/base.css           Design tokens, reset, shared components
    └── js/                    config.js, api.js (only module calling fetch), page scripts
```

The applications are separated at the top level because they are built, tested and deployed independently: Render only sees `backend/` and Vercel only serves `frontend/`. They share no code, only a contract: the JSON format of the REST API.

4.1.2 Backend Modularity: Why Each File Lives Where It Does

Each layer of Section 3.1 is a folder with a single responsibility. The guiding rule is: **a module is placed in the layer whose knowledge it needs, and no more.**

Folder	Layer	Knows about	Must NOT know about	Example
`routes/`	API / Routing	URLs, HTTP verbs, route middlewares	Business rules, SQL	`appointmentRoutes.js`
`controllers/`	API / Routing	`req`, `res`, status codes, JSON	SQL, table names	`appointmentController.js`
`services/`	Business Logic	BR-CONF-01..05, use case flow	HTTP, SQL text	`bookingService.js`
`repositories/`	Data Access	SQL, tables, transactions, locks	HTTP, business decisions	`appointmentRepository.js`
`middlewares/`	Cross-cutting	The request pipeline	Specific use cases	`errorHandler.js`, `authenticate.js`
`config/`, `utils/`	Infrastructure	Environment, DB pool, shared helpers	Domain logic	`db.js`, `AppError.js`

Dependency rule. Calls only go *downwards*: `routes → controllers → services → repositories → config/db`. A controller never runs SQL, a service never touches `req`/`res`, and only repositories import `db.js`. As a result, a business rule such as BR-CONF-01 (2-hour advance notice) exists in exactly one place and can be unit-tested without HTTP or a database; a schema change (Section 4.3) only affects repositories; and replacing the frontend (ADR 3) would not change the backend.

Request walkthrough (CU-08). The booking sequence diagram (Section 3.2.2-A) maps onto files as follows:

1. `routes/appointmentRoutes.js` matches `POST /api/appointments` and runs the authentication middleware.
2. `controllers/appointmentController.js` reads `{ slotId, sessionType, inquiry }` and the student ID from the JWT, and calls `bookingService.createAppointment(...)`.
3. `services/bookingService.js` applies BR-CONF-01, 04 and 05; if a rule fails it throws `new AppError(409, 'This time slot has just been reserved by another student.')`.
4. `repositories/appointmentRepository.js` runs `BEGIN`, `SELECT ... FOR UPDATE`, `UPDATE availability_slots`, `INSERT INTO appointments`, `COMMIT` inside `db.withTransaction()`.
5. The controller answers `201 Created`; on any error the transaction rolls back and the error middleware sends the JSON error.

The skeleton already contains one complete vertical slice, the health check (`healthRoutes.js → healthController.js → healthService.js → healthRepository.js`), which every team member uses

as the template for new use cases.

Why layers and not feature folders. The alternative is one folder per feature (`booking/`, `auth/`) containing all layers. We chose layer folders because (a) Section 3.1 and NFR-04 define the architecture in terms of layers, so the tree makes it visible and checkable; (b) with 14 use cases each layer folder stays small; and (c) the design patterns of Section 4.2 are layer-wide (Repository in `repositories/`, Chain of Responsibility in `middlewares/`). Inside each layer, files are named after their feature, so finding the code of one use case stays easy.

`app.js` vs `server.js`. `app.js` configures Express but does not open a port; `server.js` calls `listen()` and handles `SIGTERM` (sent by Render on every deploy) by closing the server and the database pool. The separation lets tests send requests to the app in memory with Supertest.

4.1.3 Dependency Management with npm

Maven and Gradle declare libraries, download them from a central repository and fix their versions. In Node.js this is done by **npm** and the npm registry, through two committed files: `package.json` and `package-lock.json`. The downloaded code (`node_modules/`) is never committed (`.gitignore`); it is rebuilt from those files.

`package.json`, the manifest. Written by the team, it declares metadata, the required runtime (`"engines": { "node": ">=22" }`), `"private": true` (prevents accidental publishing) and the **scripts**, which play the role of Maven goals or Gradle tasks. Developers, CI and Render run the same commands:

```
"scripts": {
  "start": "node src/server.js",      "dev": "nodemon src/server.js",
  "lint": "eslint .",                 "test": "jest",
  "test:coverage": "jest --coverage"
}
```

`dependencies` vs `devDependencies`. The criterion is: *does the running application need the package in production?*

Package	Type	Purpose
`express`	dependency	HTTP framework (ADR 1)
`pg`	dependency	PostgreSQL driver, parameterized queries (ADR 2, NFR-02)
`cors`, `cookie-parser`	dependency	Cross-site requests with the HttpOnly JWT cookie (ADR 4)
`jsonwebtoken`	dependency	Signing and verifying JWTs (CU-02)
`dotenv`	dependency	Loads `.env` in local development
`eslint`, `@eslint/js`, `globals`	devDependency	Static analysis (NFR-04)
`jest`, `supertest`	devDependency	Unit and HTTP integration tests (NFR-03, NFR-04)

Package	Type	Purpose
`nodemon`	devDependency	Auto-restart during development

Developers and CI install both groups (CI needs the linter and tests). Render installs with `npm ci --omit=dev`, so test and lint tools are never deployed, reducing size and attack surface.

Semantic Versioning. Versions are `MAJOR.MINOR.PATCH`: MAJOR may break the API, MINOR adds compatible features, PATCH only fixes bugs. The prefix sets the accepted range:

Range	Accepts	Example
`^5.3.0` (caret, npm default)	`>=5.3.0 <6.0.0`	`express`: 5.x fixes and features, never Express 6
`~5.3.0` (tilde)	`>=5.3.0 <5.4.0`	Patch updates only, for very sensitive packages
`5.3.0` (exact)	Only that version	Not needed: the lock file pins versions

For `0.x` versions the caret is stricter (`^0.4.1` means `<0.5.0`), because any `0.x` release may break. We use `^` everywhere: compatible fixes can be adopted deliberately with `npm update`, while the lock file keeps installs reproducible.

`package-lock.json`, reproducibility. A range alone could resolve differently on different days. The lock file, generated by npm and committed, records the exact version of every package in the tree, including transitive ones (our 12 direct dependencies expand to several hundred packages). For each one it stores the `version`, the `resolved` download URL, an `integrity` SHA-512 hash (npm rejects a package whose content does not match, protecting against tampering) and `dev: true` when it is only needed for development. The two install commands use it differently:

- `npm install` (developers): resolves ranges and **updates** the lock file. Used to add packages (`npm install pg`, `npm install -D jest`).
- `npm ci` (CI and Render): installs **exactly** what the lock file says and fails if it disagrees with `package.json`, so the code tested in CI is the code that runs in production.

Dependency security (NFR-02). CI runs `npm audit --omit=dev --audit-level=high` and fails on high or critical vulnerabilities in production packages (currently 0).

4.1.4 Core Integration

- **Configuration (`config/env.js`).** The only module that reads `process.env`. It gives defaults for development and **fails at startup in production** if `DATABASE_URL`, `JWT_SECRET` or `FRONTEND_ORIGIN` is missing. Secrets live in `.env` (ignored by Git) or in the Render dashboard; `.env.example` documents them.
- **Database connection (`config/db.js`, ADR 2).** One shared `pg.Pool`, so connections are reused (NFR-01). It exposes `query(text, params)`, always with `$1, $2...` placeholders (NFR-02), and `withTransaction(work)`, which runs `BEGIN`, the work, then `COMMIT`, or `ROLLBACK` if anything throws,

always releasing the client. It is the base for the booking transaction with `SELECT ... FOR UPDATE` (CU-08, G2, NFR-03) and the suspension transaction (CU-13).

- **Error handling.** Services throw `AppError(status, message, code)`. Express 5 forwards rejected promises from `async` handlers automatically, so controllers need no `try/catch`. `notFound.js` turns unknown routes into 404s, and `errorHandler.js` returns one JSON format, `{ "error": { "status", "message", "code" } }`, hiding internal details of 5xx errors in production.
- **Cross-site authentication (ADR 4).** Frontend (Vercel) and API (Render) are on different sites and the JWT travels in an HttpOnly cookie. The skeleton enables `cors({ origin: FRONTEND_ORIGIN, credentials: true })` (other origins are blocked), `cookie-parser` (the auth middleware reads `req.cookies`) and `trust proxy` (real client IP behind Render's proxy, used by the audit logs of CU-14).
- **Frontend client (`frontend/js/api.js`).** No page calls `fetch()` directly. The module adds `credentials: 'include'` so the browser sends the cookie while JavaScript never sees the token; serialises JSON; adds the `X-Requested-With` header, which cannot be sent cross-site without passing CORS and supports the CSRF protection of ADR 4; turns error responses into an `ApiError`; and redirects to the login page on `401` (CU-11, A1). Usage: `await api.get('/tutors/' + id + '/availability')`. `config.js` selects the API URL (local or Render) without a build step (ADR 3).
- **Base CSS (`frontend/css/base.css`).** Design tokens as CSS variables (colours, typography, spacing), a reset and shared components (cards, buttons, form fields, badges, toasts), including the slot colours for FullCalendar (ADR 5): available slots in green (CU-04, CU-07).

4.1.5 Quality Assurance, Deployment and Workflow

- **ESLint** (`eslint.config.js`, flat configuration required since ESLint 9): recommended rules plus `prefer-const`, `no-var` and `eqeqeq`.
- **Jest + Supertest**: tests for the health endpoints, 404 and malformed JSON handling, the CORS policy, and `withTransaction` commit/rollback. The database is mocked, so the suite runs without PostgreSQL. Coverage threshold: 70 % of lines (NFR-04); current coverage is about 95 %.
- **GitHub Actions** (`ci.yml`): on every push and Pull Request to `main` it runs `npm ci → lint → test:coverage → npm audit`. A red check blocks the merge.
- **Deployment (ADR 7).** `render.yaml` defines the `unimentor-api` web service (`rootDir: backend`, `npm ci --omit=dev`, `npm start`, health check `/api/health`) and the `unimentor-db` PostgreSQL database; `DATABASE_URL` is injected from the database and `JWT_SECRET` is generated by Render. Vercel serves `frontend/` as static files over its CDN. Both redeploy on every push to `main`. The landing page calls `/api/health` and `/api/health/db` through `api.js`, showing whether the whole chain (browser → Vercel → Render API → PostgreSQL) works.
- **Workflow.** Each use case gets its own branch (`feature/cu-08-booking`) and Pull Request, reviewed by another member and merged only with a green CI. A new use case adds its four files (route, controller, service, repository), mounts its router in `routes/index.js` and adds tests, following the health module. The Git history therefore records each member's work, and `main` always passes linting and tests.